

Un solo server, molti siti

Nginx com'è appena installato, e come lo lasciano gli esercizi. Ogni blocco qui sotto viene da un lab acceso: l'albero dei processi, i socket che tiene, la configurazione che accetta e le richieste a cui ha risposto.

Comando `qlab run nginx-lab`

VM `nginx-lab`, porta 80 inoltrata su una porta host dinamica

Credenziali `labuser` / `labpass`

Esito `qlab test nginx-lab` → 6 esercizi, 36 controlli, tutti superati

1. Cosa sta girando

```
nginx version: nginx/1.18.0 (Ubuntu)
active
enabled
```

```
root      1605      1 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
www-data  2230     1605 nginx: worker process
```

Due processi, e la divisione fra loro è tutta l'architettura. Il **master** gira come `root` — deve, perché legarsi alla porta 80 richiede privilegi ed è lui a rileggere la configurazione. Il **worker** gira come `www-data` ed è il processo che parla davvero con la rete. Un difetto nella gestione delle richieste finisce quindi in un processo senza privilegi, ed è esattamente il senso di questa divisione.

È anche il motivo per cui `systemctl reload nginx` non è come `restart`: il master tiene i socket in ascolto e passa ai worker una configurazione nuova, senza far cadere nessuna connessione.

```
LISTEN 0      4096      127.0.0.53%lo:53      0.0.0.0:*      users:((("systemd-resolve",pid=210,fd=14))
LISTEN 0      511        0.0.0.0:80      0.0.0.0:*      users:((("nginx",pid=2230,fd=6),
("nginx",pid=1605,fd=6))
LISTEN 0      128        0.0.0.0:22      0.0.0.0:*      users:((("sshd",pid=1664,fd=3))
LISTEN 0      511        [::]:80      [::]:*      users:((("nginx",pid=2230,fd=7),
("nginx",pid=1605,fd=7))
LISTEN 0      128        [::]:22      [::]:*      users:((("sshd",pid=1664,fd=4))
```

2. Il sito con cui si avvia

```
total 0
lrwxrwxrwx 1 root root 34 Sep 14 09:27 default -> /etc/nginx/sites-available/default

default
```

`sites-available` contiene ciò che esiste; `sites-enabled` ciò che è acceso, sotto forma di collegamenti simbolici. Disattivare un sito è togliere un link, non cancellare un file — ed è il motivo per cui le due cartelle esistono.

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    root /var/www/html;
    index index.html index.htm index.nginx-debian.html;
    server_name _;
    location / {
        try_files $uri $uri/ =404;
    }
}
```

Qui vanno nominate tre cose. `default_server` fa di questo il sito che risponde quando nient'altro corrisponde. `server_name` non è un carattere jolly con un significato: è deliberatamente un hostname non valido, così questo blocco riceve richieste solo in quanto predefinito. E `try_files $uri $uri/ =404` dice: prova il file, poi la directory, poi arrenditi onestamente, invece di ripiegare su qualcosa di non voluto.

3. Nginx non carica una configurazione che non gli piace

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

`nginx -t` analizza l'intero albero e risponde prima che qualcosa vada in produzione. È il comando da lanciare prima di ogni reload; il guasto che previene è il refuso che manda giù il server nel momento in cui si ricarica.

4. Le richieste, e dove finisce un nome sconosciuto

```
$ curl -I http://localhost/
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Mon, 14 Sep 2026 09:32:12 GMT
Content-Type: text/html
Content-Length: 19
Last-Modified: Mon, 14 Sep 2026 09:31:55 GMT

$ curl -H "Host: mysite.local" http://localhost/
<h1>Test Page</h1>
```

La seconda richiesta è quella interessante. Chiede `mysite.local`, un nome per cui questo server non ha alcun sito — gli esercizi creano quel virtual host e poi lo rimuovono. Non trovando corrispondenza, la richiesta viene servita dal blocco marcato `default_server`, ed è per questo che torna la pagina predefinita invece di un errore.

Su un server vero è una trappola da conoscere: un hostname puntato al vostro indirizzo che non avete mai configurato non viene rifiutato, riceve quello che serve il vostro sito predefinito.

```
127.0.0.1 - - [14/Sep/2026:09:32:10 +0000] "GET /secret/ HTTP/1.1" 401 188 "-" "curl/7.81.0"
127.0.0.1 - admin [14/Sep/2026:09:32:10 +0000] "GET /secret/ HTTP/1.1" 200 16 "-" "curl/7.81.0"
127.0.0.1 - admin [14/Sep/2026:09:32:10 +0000] "GET /secret/ HTTP/1.1" 200 16 "-" "curl/7.81.0"
127.0.0.1 - admin [14/Sep/2026:09:32:10 +0000] "GET /secret/ HTTP/1.1" 401 188 "-" "curl/7.81.0"
127.0.0.1 - - [14/Sep/2026:09:32:12 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/7.81.0"
127.0.0.1 - - [14/Sep/2026:09:32:12 +0000] "GET / HTTP/1.1" 200 19 "-" "curl/7.81.0"
```

Ogni riga di `access.log` è una richiesta: indirizzo, ora, riga di richiesta, stato, byte, referrer e user agent. Il formato lo decide `log_format` e quello predefinito si chiama `combined`.

5. Verifica

```
Exercise 1: All 9 checks passed
Exercise 2: All 6 checks passed
Exercise 3: All 4 checks passed
Exercise 4: All 7 checks passed
Exercise 5: All 4 checks passed
Exercise 6: All 6 checks passed
Exercises run:      6
Exercises passed:   6
Exercises failed:   0
Exercises skipped:  0
All exercises passed!
```

I sei esercizi costruiscono e smontano ciò che questo documento mostra solo a riposo: un secondo virtual host, un reverse proxy davanti a un'applicazione locale, l'analisi dei log e le prime intestazioni di sicurezza.

6. Cosa portarsi via

- Master come root, worker come `www-data` : privilegi solo dove servono.
- `reload` rilegge la configurazione senza far cadere connessioni; `restart` le fa cadere.
- `sites-available` e `sites-enabled` separano ciò che *esiste* da ciò che è *attivo*, con un symlink come interruttore.
- `default_server` raccoglie ogni nome che non avete configurato. Non è un percorso d'errore, è un ripiego: guardate cosa serve.
- Lanciate `nginx -t` prima di ogni reload.

`guide.md` del plugin porta gli esercizi: virtual host, reverse proxy, lettura dei log e le prime intestazioni di sicurezza.